UNIVERSIDADE FEDERAL DE ITAJUBÁ

Classificação de Retinopatia Diabética no Raspberry Pi 5

Relatório técnico do Projeto Final (Parte 1) da disciplina IESTI05 – Edge AI Engineering, ministrada pelo Prof. Marcelo José Rovai

Mirela Vitória Domiciano

ITAJUBÁ

2026

SUMÁRIO

1 Introdução....4
2 Objetivos....4
3 Trabalhos relacionados....5
4 Dataset e preparação....5
4.1 APTOS 2019....5
4.2 Deduplicação....6
4.3 Divisão treino / validação / teste....8
4.4 Pré-processamento....9
4.5 Aumento de dados e balanceamento....9
5 Modelos e treinamento....10
5.1 Arquiteturas....10
5.2 Treino em duas fases....10
5.3 Seleção do modelo e limiares....11
6 Otimização para a borda....11
6.1 Conversão para TFLite....11
6.2 Variantes de quantização....11
7 Avaliação de desempenho....12
7.1 Métricas....12
7.2 Modelos Keras....13
7.3 Modelos quantizados no teste....14
7.4 Benchmark no Raspberry Pi 5....15
7.5 Análise de erros....16
8 Arquitetura do sistema....17
8.1 Diagrama de arquitetura....17
8.2 Aplicação....17
8.3 Tratamento de erros....18
9 Hardware e instalação....18
9.1 Montagem do hardware....18
9.2 Software e Testes....19
10 Dificuldades e soluções....20
11 Limitações e trabalhos futuros....20
12 Conclusão....21
REFERÊNCIAS....22

RESUMO

Este trabalho apresenta um sistema que classifica retinografias na escala ICDR de cinco graus e indica se o caso é referível (grau ≥ 2), com toda a inferência feita localmente num Raspberry Pi 5. Foram treinadas ResNet-50 e MobileNetV2 no APTOS 2019, após deduplicação por hash perceptual, em 224 e 384 px. No teste, a MobileNetV2 a 384 px atingiu QWK de 0,895 e AUC de referível de 0,974, empatando com a ResNet-50 (0,901 e 0,979) com cerca de 10 vezes menos parâmetros.

Para a borda, o modelo foi convertido para TFLite em três variantes. A float16 (4,5 MB) não perde desempenho; a INT8 completa (2,7 MB) perde 0,03 de QWK e mantém a AUC, mas desloca as probabilidades e reduz a sensibilidade de 0,93 para 0,84 com o corte calibrado no modelo float. A aplicação Flask recebe imagens da câmera, de upload ou de amostras do test set. No Raspberry Pi 5, o MobileNetV2 quantizado em INT8 (2,7 MB) processa 41,3 imagens por segundo ponta a ponta, o MobileNetV2 float16 processa 21,9 e o ResNet-50 com quantização dinâmica processa 6,8, todos com temperatura abaixo de 70 °C.

Palavras-chave: retinopatia diabética; edge AI; quantização; Raspberry Pi; redes neurais convolucionais.

1 Introdução

A retinopatia diabética é uma das principais causas de cegueira evitável em adultos, e a perda de visão pode ser prevenida quando a doença é detectada cedo. Para isso, pessoas com diabetes precisam de exames de fundo de olho periódicos, mas a avaliação depende de oftalmologistas, que são escassos fora dos grandes centros. A gravidade é graduada pela escala ICDR em cinco níveis: 0 (sem retinopatia), 1 (leve), 2 (moderada), 3 (severa) e 4 (proliferativa). Na triagem, a decisão prática é binária: a partir do grau 2 o paciente deve ser encaminhado ao especialista (retinopatia referível).

Rodar a classificação na borda atende a três necessidades da triagem fora dos centros: funcionar sem conexão estável, não enviar imagens de pacientes para a nuvem e caber em hardware barato. O objetivo do projeto é mostrar que um modelo com desempenho próximo ao do estado da arte para o APTOS roda num Raspberry Pi 5 em tempo útil.

O projeto é o recorte inicial do TCC "Triagem de Retinopatia Diabética em Edge AI Fundamentada em Validação Cross-Domain". Ficam fora deste trabalho, por decisão de escopo: a validação em outro domínio (Messidor-2, nunca usado aqui), a adaptação de domínio e o modelo de linguagem explicativo.

2 Objetivos

O objetivo geral é desenvolver e avaliar um sistema embarcado que classifique a retinopatia diabética nos cinco graus da escala ICDR e indique a necessidade de encaminhamento, com a inferência executada localmente num Raspberry Pi 5. Os objetivos específicos são:

- construir, a partir do APTOS 2019, um conjunto de dados sem duplicatas e dividido de forma estratificada;
- treinar e comparar a ResNet-50 e a MobileNetV2 em duas resoluções de entrada, com seleção pelo QWK de validação;
- avaliar o efeito de diferentes estratégias de quantização sobre as métricas de graduação e de triagem;
- implementar uma aplicação Flask com câmera ao vivo, tratamento de erros e testes unitários;
- medir latência, taxa de quadros e uso de recursos no Raspberry Pi 5, atendendo aos requisitos da disciplina (acurácia de pelo menos 70% e pelo menos 0,5 FPS).

3 Trabalhos relacionados

Redes convolucionais já alcançam desempenho comparável ao de especialistas na detecção de retinopatia referível. Gulshan et al. (2016) treinaram uma Inception-v3 com cerca de 128 mil imagens graduadas por oftalmologistas e relataram alta sensibilidade e especificidade em dois conjuntos de validação. Ting et al. (2017) confirmaram o resultado em populações multiétnicas, e o IDx-DR (Abràmoff et al., 2018) foi o primeiro sistema autônomo de diagnóstico por IA aprovado pela FDA, justamente para triagem de retinopatia.

Duas competições do Kaggle tornaram o problema um benchmark público: a de 2015, com imagens do EyePACS, e a APTOS 2019, com imagens do Aravind Eye Hospital, na Índia. Ambas usam o kappa ponderado quadrático (QWK; Cohen, 1968) como métrica, porque ele penaliza mais os erros entre graus distantes, o que combina com a natureza ordinal da escala ICDR (Wilkinson et al., 2003). As soluções de topo dessas competições usaram ensembles de redes grandes em alta resolução, fora do alcance de um dispositivo de borda.

Este trabalho compara dois extremos de custo: a ResNet-50 (He et al., 2016), referência em transfer learning, e a MobileNetV2 (Sandler et al., 2018), projetada para dispositivos móveis com convoluções separáveis em profundidade. Para a implantação, usa-se a quantização inteira de Jacob et al. (2018), base do TensorFlow Lite, e o fluxo de classificação de imagens no Raspberry Pi descrito pelo professor Rovai no livro da disciplina. O diferencial em relação a esse fluxo é a avaliação da quantização com métricas clínicas, e não só acurácia.

4 Dataset e preparação

4.1 APTOS 2019

O APTOS 2019 Blindness Detection tem 3.662 retinografias rotuladas por médicos na escala ICDR, coletadas no Aravind Eye Hospital em várias clínicas e câmeras. Foi usada a versão redimensionada para 512 px, suficiente para as resoluções de entrada testadas (224 e 384 px). A distribuição é desbalanceada: metade das imagens é grau 0, e os graus 3 e 4 juntos somam cerca de 13%.

4.2 Deduplicação

A presença de imagens duplicadas no APTOS 2019 é um problema conhecido pela comunidade que trabalha com o dataset. Sikder et al. (2021) relataram que ele contém imagens duplicadas, a maioria com rótulos diferentes, mas buscaram apenas cópias com a mesma resolução; o método usado aqui detecta também cópias redimensionadas ou recomprimidas. A mesma retina aparece mais de uma vez, nem sempre como arquivo idêntico: as cópias podem ter resolução, recorte ou compressão diferentes. Isso é um problema porque a divisão em treino, validação e teste é feita por imagem: cópias da mesma retina podem cair em conjuntos diferentes, e o modelo passa a ser avaliado em imagens que já viu.

Para encontrar as cópias, cada imagem recebeu um hash MD5, que detecta arquivos idênticos byte a byte, e um hash perceptual (pHash), que resume a aparência da imagem e continua igual quando ela é redimensionada ou recomprimida. O pHash foi calculado sobre a retina já recortada, para que diferenças nas bordas pretas ou no enquadramento não escondessem uma duplicata. Foram consideradas duplicatas apenas as imagens com pHash idêntico, comparando pares diretos, sem encadear grupos.

Foram encontrados 160 pares duplicados. Em 118 deles, envolvendo 219 imagens, as cópias têm o mesmo grau (Figura 1). Esses pares se concentram nas classes com doença: o grau 2 responde por 47% deles, e o grau 0, que é metade do dataset, por apenas 15%. É justamente nessas classes que o vazamento mais inflaria o desempenho. Nesses casos, manteve-se uma cópia por grupo.

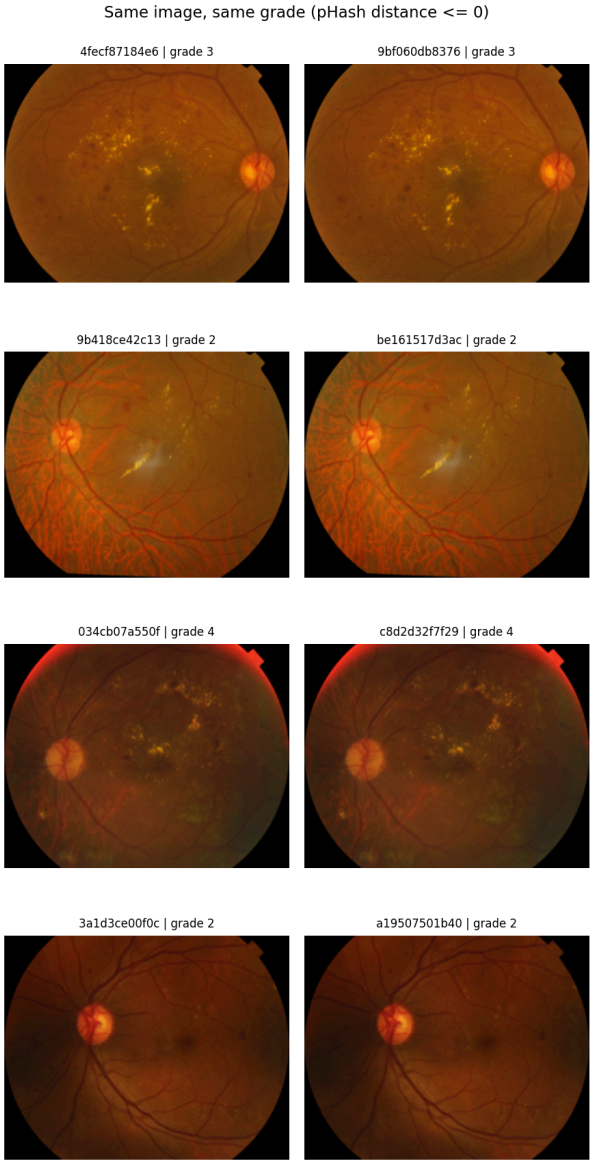
Os outros 42 pares, envolvendo 77 imagens, são a mesma retina com graus diferentes (Figura 2), o que indica ruído de rotulagem no próprio dataset. A maioria diverge em um grau, mas há casos de dois e até três graus de diferença. Como não há como saber qual rótulo está correto, as duas imagens foram removidas. Ao todo, saíram 185 imagens, e o conjunto final tem 3.477 (Tabela 1).

Tabela 1 – Resultado da deduplicação

Tipo de par	Pares	Imagens envolvidas	Imagens removidas
Mesmo grau	118	219	108
Graus diferentes	42	77	77
Total	160	—	185

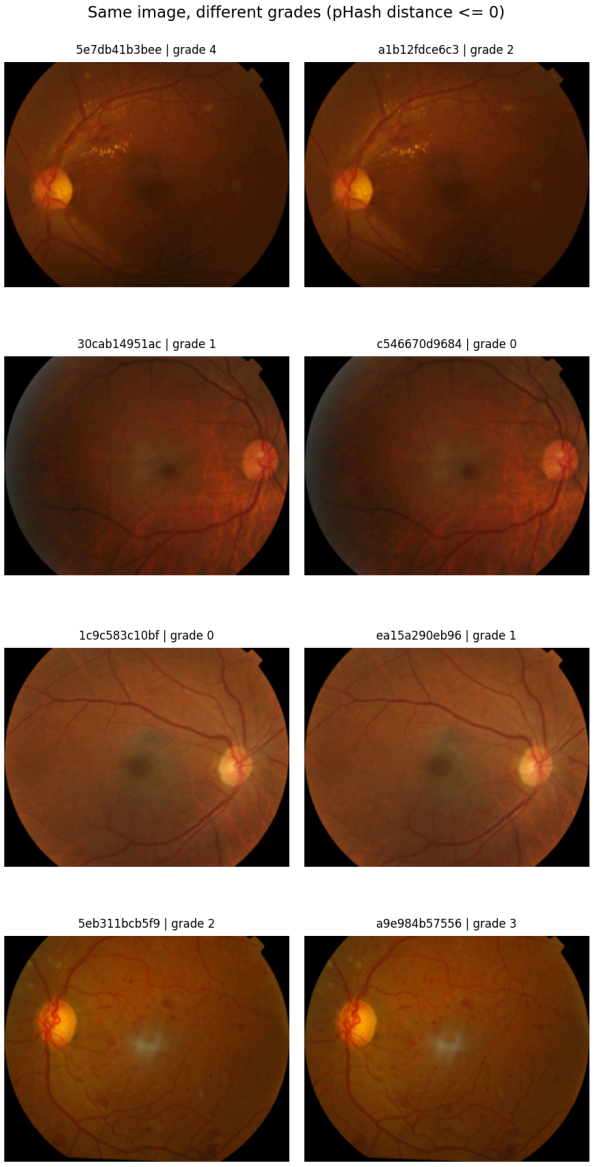
Fonte: autoria própria.

Figura 1 – Pares duplicados com o mesmo grau (distância de pHash igual a zero)



Fonte: autoria própria, a partir do APTOS 2019.

Figura 2 – Pares duplicados com graus diferentes, removidos do conjunto

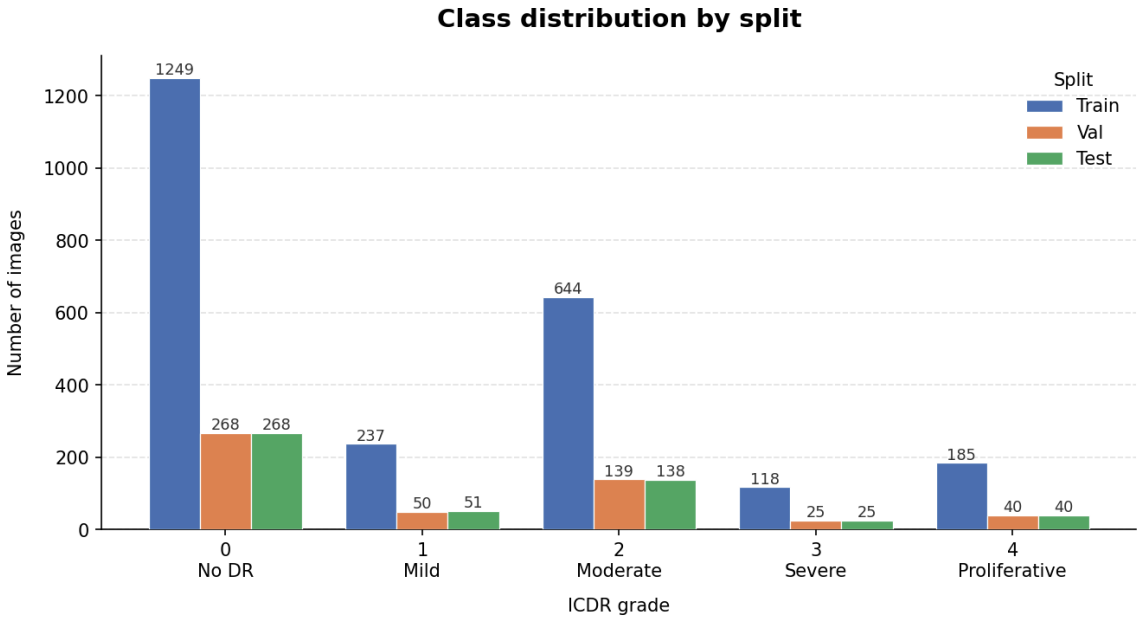


Fonte: autoria própria, a partir do APTOS 2019.

4.3 Divisão treino / validação / teste

Após a deduplicação, as imagens foram divididas em treino, validação e teste, estratificando por grau e com semente fixa (42). A validação serviu para escolher épocas, limiares e variantes de quantização; o teste foi usado uma única vez por modelo, no fim. A divisão ficou em 70% para treino e 15% para validação e teste, com a mesma proporção de graus nos três conjuntos (Figura 3).

Figura 3 – Distribuição das imagens por grau ICDR nos conjuntos de treino, validação e teste



Fonte: autoria própria.

4.4 Pré-processamento

O pré-processamento é determinístico e o mesmo código (preprocess.py, só NumPy e Pillow) roda no Colab e no Raspberry Pi. Ele localiza a retina pelo limiar de brilho, recorta a caixa que a contém, completa com preto até ficar quadrada e redimensiona com filtro Lanczos. Um teste de paridade confirma que o Pi produz, pixel a pixel, as mesmas imagens usadas na avaliação.

4.5 Aumento de dados e balanceamento

No treino, cada imagem passa por inversão horizontal e vertical, rotação livre de 360° com preenchimento preto, zoom de ±10% e brilho multiplicativo de ±10%. A retina não tem orientação canônica, então rotação e inversão não criam imagens irreais.

Para o balanceamento, cada classe recebe peso proporcional à raiz quadrada da frequência inversa, normalizado. Sem pesos, o grau 0 teria 10,6 vezes a influência do grau 3; com a raiz, 3,3 vezes. A raiz evita o exagero dos pesos inversos completos, que instabilizam o treino com classes de pouco mais de cem imagens.

Tabela 2 – Imagens e peso de cada grau no treino

Grau	Imagens no treino	Peso
0	1.249	0,68

Grau	Imagens no treino	Peso
1	237	1,57
2	644	0,95
3	118	2,23
4	185	1,78

Fonte: autoria própria.

5 Modelos e treinamento

5.1 Arquiteturas

Os dois modelos partem de pesos pré-treinados no ImageNet e têm a mesma estrutura: camada de normalização, backbone convolucional, global average pooling, dropout de 0,3 e uma camada densa softmax com cinco saídas.

Tabela 3 – Comparação das arquiteturas

	ResNet-50	MobileNetV2
Parâmetros	~23,6 M	~2,3 M
Normalização de entrada	Caffe (BGR, subtração da média)	escala para [−1, 1]
Camadas liberadas na fase 2	bloco `conv5_`	blocos 14–16 e `Conv_1`

Fonte: autoria própria.

A normalização fica dentro do modelo, como primeira camada. Assim o modelo recebe pixels crus (0–255), o que simplifica a quantização INT8 e garante que o Pi não precise replicar nenhuma conta de normalização.

5.2 Treino em duas fases

Na primeira fase, só a head treina, por 5 épocas com taxa de aprendizado 1e-3, enquanto o backbone fica congelado. Na segunda, o topo do backbone é liberado e o modelo treina por até 30 épocas com taxa menor: 1e-4 na MobileNetV2 e 1e-5 na ResNet-50. O lote tem 32 imagens, e os pesos de classe valem nas duas fases.

Foram treinadas quatro configurações: os dois modelos a 224 px e a 384 px. A troca de resolução não muda os pesos pré-treinados, só o tamanho dos mapas de ativação, o que permite isolar o efeito da resolução.

5.3 Seleção do modelo e limiares

A métrica usada para escolher o modelo é o QWK (Quadratic Weighted Kappa), a mesma da competição APTOS. Ele mede a concordância entre previsões e rótulos descontando o acaso (1 é concordância perfeita; 0, o nível de um palpite aleatório) e penaliza cada erro pelo quadrado da distância entre os graus: confundir o grau 0 com o 4 custa dezesseis vezes mais que confundir o 1 com o 2. Como os graus da escala ICDR são ordenados e a gravidade de um erro clínico depende dessa distância, o QWK é calculado na validação ao fim de cada época, e o melhor checkpoint é o de maior QWK, não o de menor loss.

Depois do treino, duas regras de decisão são ajustadas na validação e usadas sem mudança no teste e no Pi. A primeira transforma as probabilidades em um grau levando em conta a ordem entre as classes, o que aumenta o QWK em relação a simplesmente escolher a classe mais provável. A segunda define a partir de que probabilidade de doença referível o caso deve ser encaminhado.

6 Otimização para a borda

6.1 Conversão para TFLite

Os modelos Keras foram convertidos para TensorFlow Lite e executados com o runtime LiteRT, o mesmo no Colab e no Pi. Avaliar no Colab com o runtime da borda garante que os números de qualidade valem para o que roda no dispositivo.

6.2 Variantes de quantização

Para cada modelo a 384 px foram geradas quatro variantes. Na float16, os pesos ficam em meia precisão e o cálculo em float, o que reduz o arquivo pela metade. Na quantização dinâmica (dynamic range), os pesos são armazenados em INT8 e as ativações seguem em float, quantizadas em tempo de execução. Na INT8 completa, pesos e ativações são inteiros, com as faixas das ativações estimadas numa amostra de calibração; ela foi gerada com 300 imagens do treino (int8_c300) e com o treino inteiro (int8_call), para testar se a calibração era o gargalo. Cada variante foi comparada ao modelo Keras na validação, por QWK e pela fração de imagens em que prevê o mesmo grau (Tabela 4).

Tabela 4 – Variantes de quantização na validação (384 px)

Modelo	Variante	Tamanho (MB)	QWK val.	Mesmo grau que o Keras	Maior dif. de probabilidade
MobileNetV2	Keras fp32	—	0,885	100%	0
MobileNetV2	float16	4,52	0,878	99,4%	0,051
MobileNetV2	dinâmica	2,54	0,855	94,1%	0,444
MobileNetV2	int8_c300	2,74	0,840	91,6%	0,398
MobileNetV2	int8_call	2,74	0,837	90,8%	0,379
ResNet-50	Keras fp32	—	0,882	100%	0
ResNet-50	float16	47,07	0,882	100%	0,006
ResNet-50	dinâmica	23,95	0,884	99,4%	0,151
ResNet-50	int8_c300	24,27	0,711	76,4%	0,993
ResNet-50	int8_call	24,27	0,703	75,9%	0,993

Fonte: autoria própria.

Na validação, os dois modelos reagiram de forma oposta à quantização. Na ResNet-50, a variante dinâmica preserva o desempenho, mas a INT8 completa derruba o QWK de 0,88 para 0,71; como a única diferença entre as duas é quantizar as ativações, o problema está nelas, e calibrar com o treino inteiro não ajudou. Com base nisso, três variantes seguiram para o teste e o Pi: a MobileNetV2 float16 (4,5 MB, sem perda), a MobileNetV2 INT8 completa (2,7 MB, exigida pela disciplina) e a ResNet-50 dinâmica (23,9 MB, sem perda).

7 Avaliação de desempenho

7.1 Métricas

A métrica principal é o QWK (Seção 5.3), reportado de duas formas: escolhendo a classe mais provável (argmax) e com a regra de decisão ajustada na validação. Para o encaminhamento, usam-se AUC, sensibilidade e especificidade; em triagem, a sensibilidade pesa mais, porque um caso referível que passa despercebido custa mais que um encaminhamento desnecessário. Acurácia e F1 macro completam o quadro, este último mostrando o desempenho nas classes raras. Com 522 imagens na validação, o QWK oscila cerca de 0,01 entre épocas sem mudança real, então diferenças menores que essa não sustentam conclusões.

7.2 Modelos Keras

A resolução de entrada pesou mais que a arquitetura. A MobileNetV2 ganhou cerca de 0,03 de QWK ao passar de 224 para 384 px e empatou com a ResNet-50, com 10 vezes menos parâmetros. Todos os modelos superam com folga o requisito de 70% de acurácia.

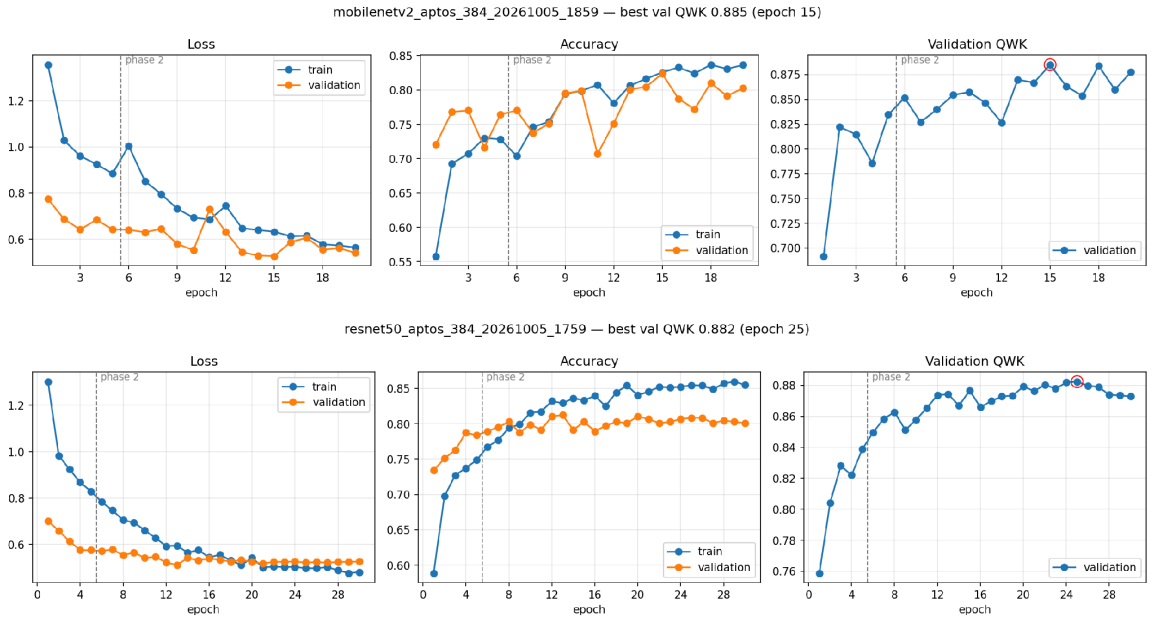
Tabela 5 – Modelos Keras no conjunto de teste

Modelo	Entrada	QWK (argmax)	QWK (limiares)	Acurácia	F1 macro	AUC referível	Sensibilidade	Especificidade
ResNet-50	384 px	0,887	0,901	0,791	0,605	0,979	0,946	0,915
MobileNetV2	384 px	0,869	0,895	0,795	0,583	0,974	0,926	0,928
ResNet-50	224 px	0,866	0,900	0,791	0,609	0,974	0,926	0,925
MobileNetV2	224 px	0,835	0,864	0,701	0,495	0,967	0,911	0,900

Fonte: autoria própria.

A Figura 4 mostra o treino a 384 px. Com taxa de 1e-4 na fase 2, a MobileNetV2 converge rápido e atinge o melhor QWK de validação (0,885) na época 15, oscilando mais entre épocas, mas sem sobreajuste. Com 1e-5, a ResNet-50 evolui de forma mais suave e chega a 0,882 na época 25; a partir da época 20 aparece um sobreajuste leve, que a seleção pelo melhor QWK de validação impede de chegar ao modelo final.

Figura 4 – Loss, acurácia e QWK por época no treino a 384 px: (a) MobileNetV2; (b) ResNet-50. A linha tracejada marca o início da fase 2 e o círculo, o melhor QWK de validação



Fonte: autoria própria.

7.3 Modelos quantizados no teste

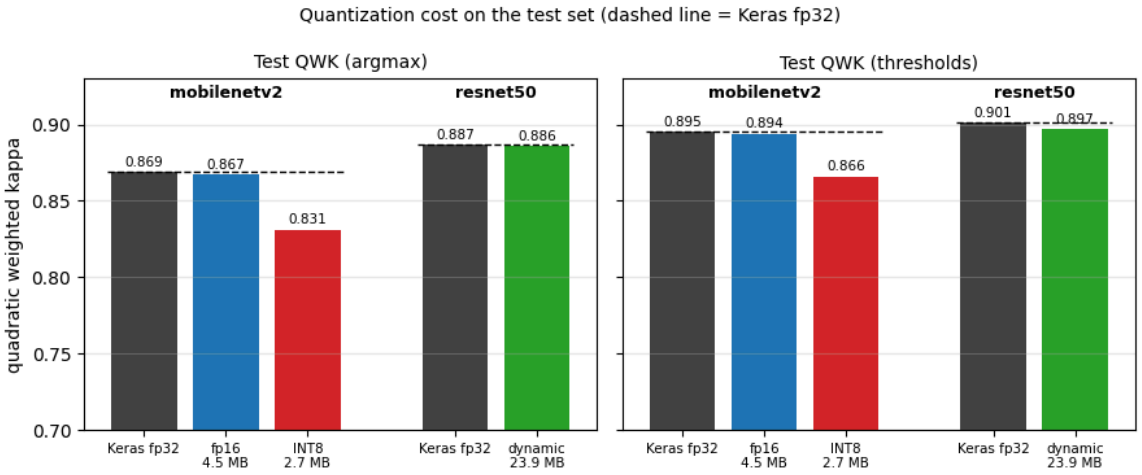
O teste confirmou o que a validação havia indicado: float16 e dinâmica não perdem desempenho, e a INT8 completa perde cerca de 0,03 de QWK.

Tabela 6 – Variantes TFLite no conjunto de teste (384 px)

Modelo	Variante	Tamanho (MB)	QWK (argmax)	QWK (limiares)	Acurácia (argmax)	AUC	Sensibilidade	Especificidade	Mesmo grau que o Keras
MobileNetV2	Keras fp32	—	0,869	0,895	0,818	0,974	0,926	0,928	100%
MobileNetV2	float16	4,52	0,867	0,894	0,812	0,974	0,926	0,928	99,0%
MobileNetV2	INT8	2,74	0,831	0,866	0,803	0,971	0,837	0,944	90,0%
ResNet-50	Keras fp32	—	0,887	0,901	0,816	0,979	0,946	0,915	100%
ResNet-50	dinâmica	23,95	0,886	0,897	0,818	0,978	0,951	0,912	99,2%

Fonte: autoria própria.

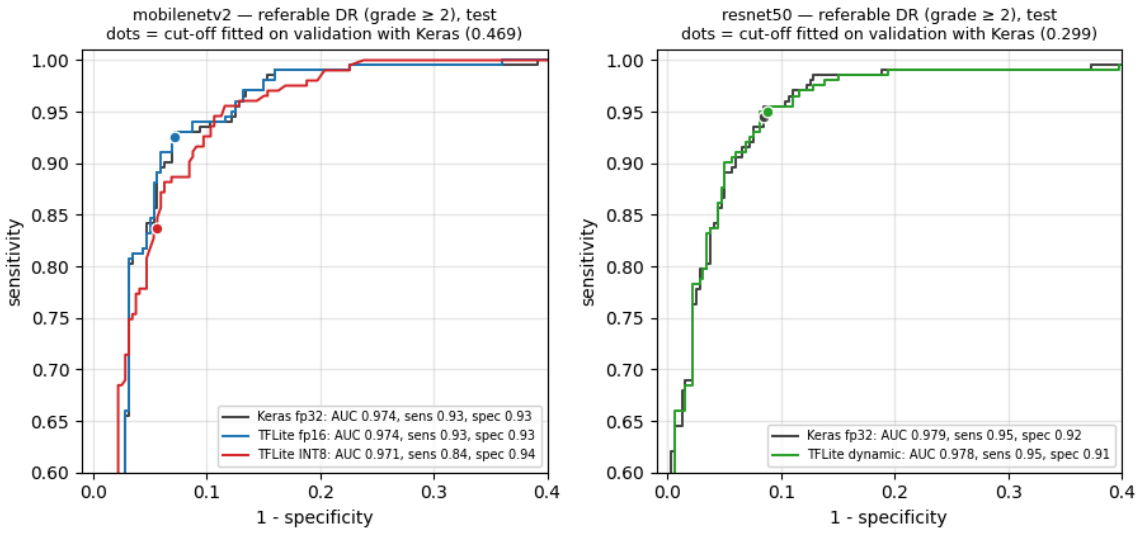
Figura 5 – QWK no teste por variante, com o modelo Keras como referência



Fonte: autoria própria.

O resultado mais relevante está na INT8. A AUC quase não muda (0,974 para 0,971), então o modelo continua ordenando bem os casos. Mas a sensibilidade cai de 0,926 para 0,837 e a especificidade sobe: as probabilidades se deslocaram, e o corte de referível calibrado no modelo float ficou mal posicionado para a INT8.

Figura 6 – Curvas ROC de retinopatia referível no teste, com o ponto de corte calibrado na validação



Fonte: autoria própria.

7.4 Benchmark no Raspberry Pi 5

A Tabela 7 apresenta o desempenho das três variantes selecionadas no Raspberry Pi 5, medido com 50 inferências por modelo, 5 repetições sobre 10 imagens, descartando a primeira inferência de aquecimento. A latência foi muito estável: o p95 ficou a menos de 2% da média em todos os modelos. A quantização INT8 reduziu a inferência do MobileNetV2 de 33,7 ms para 9,3 ms (3,6 vezes mais rápida) e o tamanho de 4,52 MB para 2,74 MB, atingindo 41,3 FPS ponta a ponta. O ganho ponta a ponta (1,9 vez) é menor que o da inferência porque o pré-processamento, de 7,3 ms, é constante entre os modelos.

O MobileNetV2 float16 opera a 21,9 FPS, adequado para a câmera ao vivo. O ResNet-50 com quantização dinâmica levou 135,6 ms por imagem (6,8 FPS), o que ainda atende à triagem, em que cada imagem de fundo de olho é analisada individualmente. A temperatura máxima foi de 67 °C, abaixo do limite de redução de frequência do processador (cerca de 80 °C), o que indica que o resfriamento ativo da case é suficiente..

Tabela 7 – Desempenho no Raspberry Pi 5

Modelo	Tamanho (MB)	Inferência média	p95	Pré-processamento	FPS ponta a ponta	CPU	Temperatura
MobileNetV2 INT8	4,52	9,3 ms	9,5 ms	7,3 ms	41,3	52%	58 °C
MobileNetV2 float16	2,74	33,7 ms	34,0 ms	7,3 ms	21,9	80%	59 °C
ResNet-50 dinâmica	23,95	135,6 ms	137,0 ms	7,3 ms	6,8	88%	67 °C

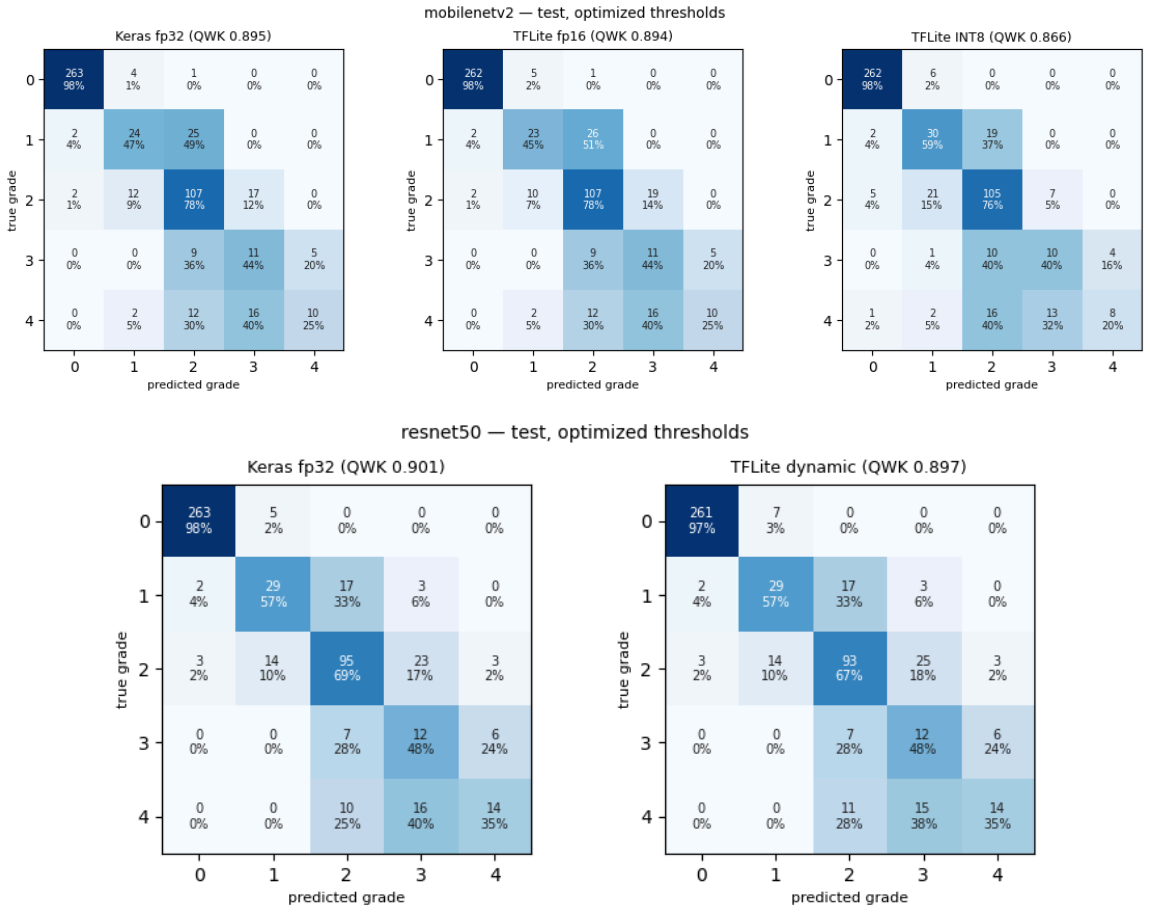
Fonte: autoria própria.

7.5 Análise de erros

Na MobileNetV2 (Figura 7a), o erro mais comum no grau 4 é a previsão como grau 3, que pesa pouco, porque os dois graus levam ao encaminhamento. Os erros relevantes são os que cruzam a fronteira entre os graus 1 e 2: o grau 1 previsto como 2 (25 de 51 imagens) gera encaminhamentos desnecessários, e o grau 2 previsto como 0 ou 1 (14 de 138) deixa casos sem encaminhamento. Na INT8, esse segundo erro sobe para 26 imagens, o que explica a queda de sensibilidade vista na Seção 7.3.

A ResNet-50 (Figura 7b) confunde menos o grau 1 com o 2 (17 de 51) e acerta mais o grau 4, mas superestima o grau 2, previsto como 3 em 23 de 138 imagens; a variante dinâmica reproduz a matriz do modelo float quase célula a célula. Com apenas 25 imagens de grau 3 e 40 de grau 4 no teste, as taxas dessas classes devem ser lidas com cautela.

Figura 7 – Matrizes de confusão no teste, com os limiares otimizados: (a) MobileNetV2 em float32, float16 e INT8; (b) ResNet-50 em float32 e dinâmica

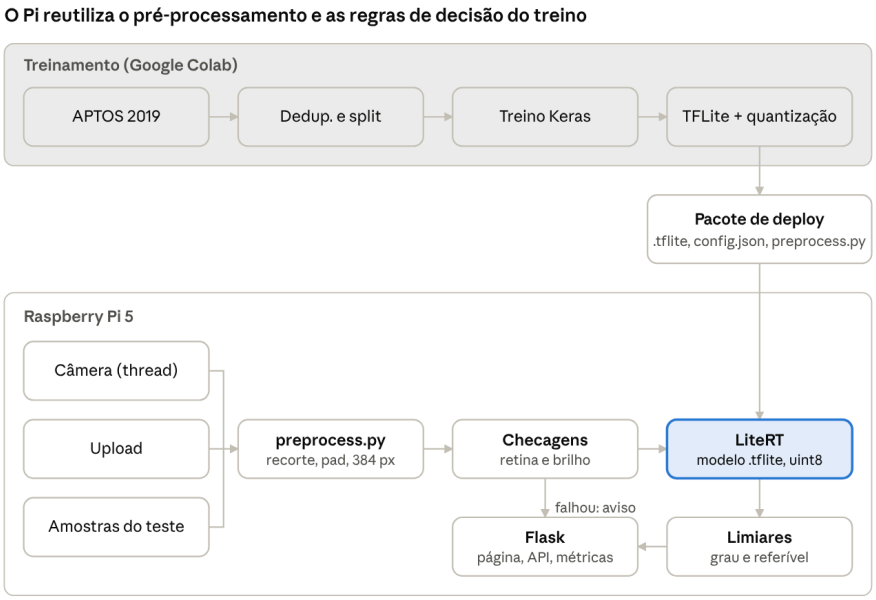


Fonte: autoria própria.

8 Arquitetura do sistema

8.1 Diagrama de arquitetura

Figura 8 – Arquitetura do sistema: treino no Google Colab e inferência no Raspberry Pi 5



Fonte: autoria própria.

O treino gera um pacote com os modelos, as regras de decisão e o mesmo `preprocess.py` usado no Colab. No Pi, toda imagem, venha da câmera, de upload ou das amostras, passa pelo mesmo caminho; se a checagem falhar, o usuário recebe um aviso em vez de um grau.

8.2 Aplicação

A aplicação tem dois módulos: o engine.py faz a inferência e não depende de Flask nem de câmera, o que permite testá-lo em qualquer máquina; o app.py cuida da interface web, da câmera e do modo benchmark. Três threads trabalham em paralelo: uma captura os quadros da câmera, outra classifica o quadro mais recente enquanto o modo ao vivo está ligado, e a do Flask serve a página, o vídeo e a API.

A página permite trocar de modelo sem reiniciar o app e mostra, ao lado do vídeo, a imagem como o modelo a vê. O modo câmera é uma simulação: a câmera fotografa retinografias exibidas num monitor, porque não há um retinógrafo disponível. Sem câmera, o app funciona por upload e pelas amostras do teste.

8.3 Tratamento de erros

Antes de chamar o modelo, o `engine.py` verifica a imagem e devolve uma mensagem clara em vez de um grau sem sentido. Nenhum desses casos interrompe a aplicação.

Tabela 8 – Casos de erro tratados pela aplicação

Caso	Como é detectado	O que o usuário vê
Retina não encontrada	o recorte não acha região clara, ou os cantos da região recortada não são escuros como numa retinografia	pedido para centralizar a imagem; nenhum grau
Imagem escura	brilho médio da metade central abaixo de 25 (escala 0–255)	pedido para aumentar o brilho da tela; nenhum grau
Confiança baixa	maior probabilidade abaixo de 0,45	o grau, com aviso para recapturar ou encaminhar
Arquivo inválido	o Pillow não consegue abrir o arquivo	erro 400 com a mensagem
Câmera ausente	Picamera2 e OpenCV falham na abertura	o app segue no modo upload e amostras

Fonte: autoria própria.

9 Hardware e instalação

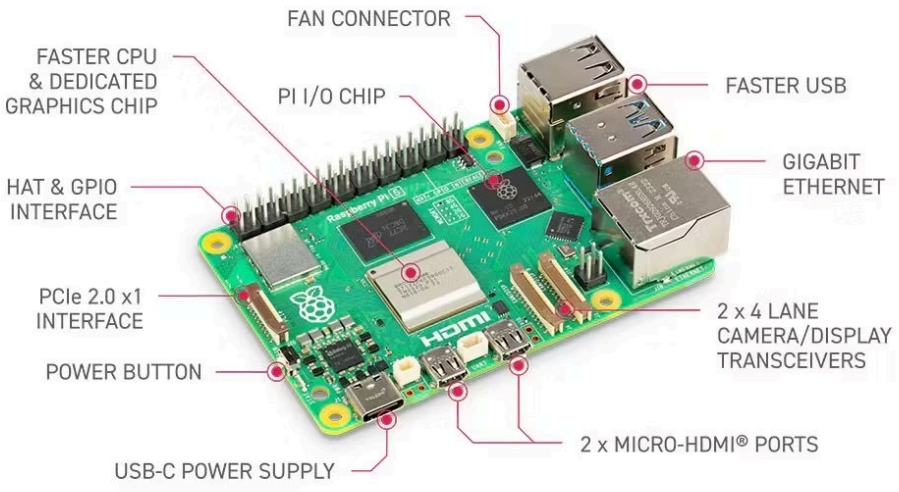
9.1 Montagem do hardware

O sistema roda num Raspberry Pi 5 com 8 GB de RAM (Figura 9). A placa tem processador quad-core Arm Cortex-A76 de 2,4 GHz, GPU VideoCore VII e duas interfaces MIPI para câmera ou display, uma das quais recebe a câmera. O Pi 5 substituiu o Raspberry Pi Zero 2 W, previsto inicialmente, com aprovação do professor.

Tabela 10 – Componentes de hardware

Componente	Especificação
Placa	Raspberry Pi 5, 8 GB de RAM
Armazenamento	cartão microSD de 64 GB
Câmera	Raspberry Pi Camera Module [PENDENTE: versão], conexão CSI
Refrigeração	case com cooler ativo
Fonte	5 V / 5 A USB-C
Tela para recaptura	monitor exibindo as retinografias

Figura 9 – Raspberry Pi 5 e seus principais componentes



Fonte: Raspberry Pi Ltd.

Para a recaptura, a câmera fica fixa diante da tela, a uma distância em que a retina ocupe a maior parte do quadro. Ambiente escuro e brilho da tela no máximo reduzem reflexos.

Figura 10 – Montagem do hardware para a recaptura de retinografias



Fonte: autoria própria.

9.2 Software e Testes

O sistema operacional é o Raspberry Pi OS Bookworm 64-bit, com Python 3.11. A Picamera2 vem do repositório do sistema, por isso o ambiente virtual é criado com acesso aos pacotes do sistema.Os 37 testes unitários rodam em qualquer máquina, com um interpretador

simulado, e cobrem o pré-processamento, a entrada do modelo, os limiares, os casos de erro, as rotas da API, o loop ao vivo e o benchmark.

10 Dificuldades e soluções

A Tabela 11 resume as principais dificuldades do projeto e como cada uma foi tratada; os detalhes estão nas seções indicadas.

Tabela 11 – Dificuldades encontradas e soluções adotadas

Dificuldade	**Solução adotada**
Imagens repetidas no APTOS, algumas com graus diferentes	Deduplicação por MD5 e pHash antes da divisão; pares conflitantes removidos (Seção 4.2)
Graus 3 e 4 com pouco mais de cem imagens no treino	Pesos de classe e seleção do modelo pelo QWK (Seções 4.5 e 5.3)
Argmax trata todos os erros entre graus como iguais	Regra de decisão ajustada na validação, que aproveita a ordem dos graus (Seção 5.3)
INT8 completa derrubou o QWK da ResNet-50	Diagnóstico: o problema está nas ativações; ResNet-50 seguiu com quantização dinâmica (Seção 6.3)
Corte de encaminhamento deslocado na INT8	Identificado pela ROC e pelas matrizes de confusão; recalibração como trabalho futuro (Seção 7.3)
Risco de normalizar a imagem duas vezes no Pi	Normalização dentro do modelo e teste unitário que verifica a entrada crua (Seção 9.3)
Pré-processamento diferente entre Colab e Pi	Um único preprocess.py nos dois lados e teste de paridade pixel a pixel (Seção 4.4)
Sem retinógrafo para o modo câmera	Recaptura de retinografias exibidas num monitor, documentada como simulação (Seção 8.2)

Fonte: autoria própria.

11 Limitações e trabalhos futuros

A principal limitação é o domínio único. Todas as imagens vêm do APTOS, e o desempenho em outras câmeras e populações não foi medido; essa é a pergunta central do TCC, que vai avaliar o modelo no Messidor-2 e testar adaptação de domínio (AdaBN). O modo câmera também não equivale a um retinógrafo: a recaptura de tela adiciona reflexo, moiré e perda de cor. Além disso, o teste tem poucas imagens de graus 3 e 4, o que torna as taxas dessas classes ruidosas.

Como trabalhos futuros, ficam a inclusão do EyePACS, dataset de retinografias bem maior que o APTOS, no treino, o que daria mais exemplos dos graus raros; a recalibração do corte de encaminhamento no modelo INT8; o treino com quantização (QAT) ou a quantização seletiva das camadas mais sensíveis; mapas de ativação (Grad-CAM) para mostrar em que

região da retina o modelo se baseia; e, na Parte 2 da disciplina, um modelo de linguagem local que explique o resultado ao profissional de saúde.

12 Conclusão

O projeto mostrou que a classificação de retinopatia diabética cabe num Raspberry Pi 5 sem perda relevante de qualidade. A MobileNetV2 a 384 px em float16, com 4,5 MB, atinge QWK de 0,894 e AUC de referível de 0,974 no teste, praticamente o mesmo que a ResNet-50. A implantação no Raspberry Pi 5 mostrou que os três modelos operam em tempo compatível com triagem: de 6,8 FPS (ResNet-50) a 41,3 FPS (MobileNetV2 INT8). A quantização INT8 tornou o MobileNetV2 3,6 vezes mais rápido, ao custo de redução na sensibilidade (0,93 para 0,84 no limiar calibrado em ponto flutuante), o que indica que a escolha entre as variantes depende do equilíbrio desejado entre velocidade e sensibilidade clínica.

Dois achados orientam o TCC. A resolução de entrada importou mais que a arquitetura. E a quantização INT8 pode preservar a AUC e ainda assim prejudicar a triagem, ao deslocar a calibração das probabilidades: avaliar um modelo quantizado só pela acurácia esconderia esse efeito.

REFERÊNCIAS

ABRÀMOFF, M. D. et al. Pivotal trial of an autonomous AI-based diagnostic system for detection of diabetic retinopathy in primary care offices. **npj Digital Medicine**, v. 1, art. 39, 2018.

COHEN, J. Weighted kappa: nominal scale agreement with provision for scaled disagreement or partial credit. **Psychological Bulletin**, v. 70, n. 4, p. 213–220, 1968.

GULSHAN, V. et al. Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. **JAMA**, v. 316, n. 22, p. 2402–2410, 2016.

HE, K. et al. Deep residual learning for image recognition. In: IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2016, Las Vegas. **Proceedings** [...]. 2016. p. 770–778.

JACOB, B. et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: IEEE/CVF CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2018, Salt Lake City. **Proceedings** [...]. 2018. p. 2704–2713.

KAGGLE. **APTOS 2019 Blindness Detection**. 2019. Disponível em: https://www.kaggle.com/competitions/aptos2019-blindness-detection. Acesso em: 6 out. 2026.

ROVAI, M. J. **[PENDENTE: título do livro da disciplina]**. Disponível em: https://mjrovai.github.io/EdgeML_Made_Ease_ebook/. Acesso em: 6 out. 2026.

SANDLER, M. et al. MobileNetV2: inverted residuals and linear bottlenecks. In: IEEE/CVF CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2018, Salt Lake City. **Proceedings** [...]. 2018. p. 4510–4520.

SIKDER, N. et al. Severity classification of diabetic retinopathy using an ensemble learning algorithm through analyzing retinal images. **Symmetry**, v. 13, n. 4, art. 670, 2021. DOI: 10.3390/sym13040670.

TING, D. S. W. et al. Development and validation of a deep learning system for diabetic retinopathy and related eye diseases using retinal images from multiethnic populations with diabetes. **JAMA**, v. 318, n. 22, p. 2211–2223, 2017.

WILKINSON, C. P. et al. Proposed international clinical diabetic retinopathy and diabetic macular edema disease severity scales. **Ophthalmology**, v. 110, n. 9, p. 1677–1682, 2003.